

Module 08

Pydantic

Type-safe data validation and serialisation for Python

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. The Problem Pydantic Solves

Python is dynamically typed. A function that expects an integer can silently receive a string and crash much later — far from where the bug was introduced. Pydantic lets you define **data models** with types. It validates data when you create an object, raises clear errors immediately, and serialises/deserialises to/from JSON automatically.

```
# Without Pydantic – fragile, silent bugs
def create_user(data: dict):
    name = data["name"]    # KeyError if missing
    age = data["age"]      # might be a string "25" instead of int 25
    # no error here – crashes later when you do arithmetic

# With Pydantic – validated at the boundary
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

user = User(name="Alice", age=25)      # OK
user = User(name="Alice", age="25")    # also OK – Pydantic coerces "25" → 25
user = User(name="Alice", age="abc")    # ValidationError: 'abc' is not an int
```

2. BaseModel — The Core

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class Article(BaseModel):
    title: str
    url: str
    word_count: int
    published_date: Optional[datetime] = None    # optional, defaults to None
    is_paywalled: bool = False                  # optional, defaults to False

# Creating from keyword arguments
article = Article(title="RBI Rate Hike", url="https://...", word_count=850)

# Creating from a dict (e.g., parsed JSON or DB row)
data = {"title": "SEBI New Rules", "url": "https://...", "word_count": 600}
article = Article(**data)
```

3. model_dump — Serialise to Dict/JSON

```
article = Article(title="Test", url="https://test.com", word_count=500)

# To a Python dict
d = article.model_dump()
# {"title": "Test", "url": "https://test.com", "word_count": 500,
#  "published_date": None, "is_paywalled": False}

# Exclude None values
```

```

d = article.model_dump(exclude_none=True)
# {"title": "Test", "url": "https://test.com", "word_count": 500, "is_paywalled": False}

# Include only specific fields
d = article.model_dump(include={"title", "url"})
# {"title": "Test", "url": "https://test.com"}

# To JSON string
json_str = article.model_dump_json()

```

4. Field — Defaults, Descriptions, Validation

```

from pydantic import BaseModel, Field

class Idea(BaseModel):
    angle: str = Field(..., min_length=10, max_length=500) # required, constrained
    score: float = Field(default=5.0, ge=0.0, le=10.0) # 0-10 range
    platform: str = Field(default="linkedin",
                          description="Target publishing platform")
    tags: list[str] = Field(default_factory=list) # mutable default!
    reasoning: str = Field(default="", alias="agent_reasoning") # DB column alias

# !! Never use mutable defaults directly – use default_factory:
# BAD: tags: list[str] = [] # shared across all instances!
# GOOD: tags: list[str] = Field(default_factory=list)

```

5. Nested Models

```

from pydantic import BaseModel
from typing import Optional

class StructuredSummary(BaseModel):
    story_narrative: str
    key_data_points: list[str]
    mechanism: str
    implications: str
    content_angles: list[str]

class RawContent(BaseModel):
    title: str
    url: str
    full_text: str
    structured_summary: Optional[StructuredSummary] = None

# Nested creation – Pydantic validates recursively
content = RawContent(
    title="RBI Rate Hike",
    url="https://...",
    full_text="...",
    structured_summary=StructuredSummary(
        story_narrative="RBI raised rates by 25 bps...",
        key_data_points=["25 bps hike", "6th consecutive hike"],
        mechanism="Inflation above 6% target",
        implications="EMIs will rise",
    )
)

```

```

        content_angles=["Impact on home loans", "How to protect savings"],
    )
)
# Serialises all nested objects correctly
print(content.model_dump_json(indent=2))

```

6. Enums — Constrained String Values

```

from enum import Enum
from pydantic import BaseModel

class Platform(str, Enum):
    LINKEDIN = "linkedin"
    TWITTER = "twitter"
    BLOG = "blog"
    EMAIL = "email"

class ApprovalStatus(str, Enum):
    PENDING = "pending_approval"
    APPROVED = "approved"
    REJECTED = "rejected"

class Idea(BaseModel):
    platform: Platform # must be one of the enum values
    approval_status: ApprovalStatus # same

# Usage
idea = Idea(platform="linkedin", approval_status="approved") # strings auto-convert
idea.platform # Platform.LINKEDIN
idea.platform.value # "linkedin" (for DB/JSON insertion)

# This raises ValidationError:
idea = Idea(platform="instagram", approval_status="approved")

```

7. model_construct — Skip Validation (Use With Care)

`model_construct()` creates a model instance WITHOUT running validators. Use ONLY when you trust the data source completely (e.g. reading from your own DB that already enforces constraints).

```

# Slow – validates everything (use for untrusted input)
user = User(name="Alice", age=25)

# Fast – skips validation (use for trusted DB data)
user = User.model_construct(name="Alice", age=25)

```

Tip: `model_construct` skips ALL validation. Only use it for data you completely trust — wrong data will silently produce invalid model instances that crash later.

8. Validators — Custom Validation Logic

```

from pydantic import BaseModel, field_validator

class Article(BaseModel):

```

```

url: str
word_count: int

@field_validator("url")
@classmethod
def url_must_start_with_https(cls, v: str) -> str:
    if not v.startswith("https://"):
        raise ValueError("URL must start with https://")
    return v

@field_validator("word_count")
@classmethod
def word_count_positive(cls, v: int) -> int:
    if v < 0:
        raise ValueError("word_count must be non-negative")
    return v

```

9. How ContentAutomation2 Uses Pydantic

Every database table has corresponding Pydantic models in `backend/app/db/models.py`. The pattern is:

- — full DB row (includes `id`, `created_at`, DB-generated fields)
- Create — minimal fields needed to INSERT a new row
- Update — all optional fields for PATCH updates

This means the API can validate incoming requests, the agents can produce structured output, and the DB layer can serialise cleanly — all with the same model.

10. Practice Exercises

- Exercise 1: Define a `BlogPost` model with: `title` (str), `body` (str), `tags` (list[str]), `published` (bool default False).
- Exercise 2: Add Field constraints: `title` 10-200 chars, `tags` max 10 items.
- Exercise 3: Add a nested `Author` model with `name` and `email` fields.
- Exercise 4: Add a `field_validator` that ensures the email ends in '@example.com'.
- Exercise 5: Add a `Status` enum: `DRAFT`, `PUBLISHED`, `ARCHIVED`.
- Exercise 6: Create a `BlogPostCreate` model (only `title`+`body` required) and a `BlogPost` model (adds `id`, `created_at`, `status`).
- Exercise 7: Serialise a `BlogPost` to JSON. Parse it back with `model_validate_json()`.